

Практическое занятие 2

Сборка, публикация приложения и организация непрерывной интеграции

Предметная область — «Бюро переводов». Платформа — .NET 10.0, система управления версиями Git, служба непрерывной интеграции GitHub Actions.

Продолжительность занятия — 90 минут.

Цель занятия

Целью занятия является приобретение практических навыков сборки и публикации приложения средствами интерфейса командной строки .NET, размещения исходного кода в удалённом репозитории системы управления версиями, а также настройки конвейера непрерывной интеграции, обеспечивающего автоматическую сборку, выполнение модульных тестов и формирование пакета развёртывания при каждом изменении исходного кода.

Постановка задачи

Требуется разместить решение «Бюро переводов» в удалённом репозитории и настроить конвейер непрерывной интеграции, обеспечивающий выполнение следующих действий при каждом изменении исходного кода:

- восстановление зависимостей решения;
- сборку решения в конфигурации **Release** под управлением операционных систем Windows и Linux;
- выполнение модульных тестов с формированием отчёта о результатах;
- публикацию Web-приложения в виде пакета развёртывания;
- сохранение полученного пакета в качестве артефакта сборки;
- отображение состояния последней сборки на титульной странице репозитория.

Результаты, которые должны быть достигнуты к завершению занятия

К завершению занятия обучающийся должен:

- понимать различие между командами **dotnet build** и **dotnet publish**, а также между развёртыванием, зависящим от платформы, и самодостаточным развёртыванием;
- уметь размещать решение в удалённом репозитории с корректно настроенным файлом исключений **.gitignore**;
- уметь составлять описание конвейера непрерывной интеграции на языке YAML;
- уметь анализировать протокол выполнения конвейера и устранять причины неуспешной сборки.

Понятие непрерывной интеграции

Непрерывная интеграция представляет собой практику разработки программного обеспечения, при которой изменения исходного кода, вносимые участниками проекта, регулярно объединяются в общей ветви репозитория, причём каждое объединение сопровождается автоматической сборкой и выполнением набора тестов.

Основным назначением непрерывной интеграции является раннее обнаружение ошибок. Ошибка, допущенная при внесении изменений, обнаруживается в течение нескольких минут после отправки изменений в репозиторий, а не на этапе приёмочных испытаний. Дополнительным результатом является формирование воспроизводимого пакета развёртывания: сборка выполняется в заведомо чистом

окружении, что исключает зависимость результата от особенностей настройки рабочего места разработчика.

Служба GitHub Actions выполняет описание конвейера, размещённое в файле формата YAML в каталоге `.github/workflows` репозитория. Описание конвейера состоит из следующих элементов:

- **события** (`on`) — условия, при наступлении которых конвейер запускается;
- **задания** (`jobs`) — независимые последовательности действий, выполняемые, как правило, параллельно;
- **шаги** (`steps`) — отдельные действия внутри задания: вызов готового действия либо выполнение команды операционной системы.

Предварительные требования

На рабочем месте должно быть установлено следующее программное обеспечение:

- пакет .NET 10.0 SDK;
- система управления версиями Git;
- учётная запись на портале GitHub.

Проверка установленного программного обеспечения выполняется следующими командами.

```
dotnet --version  
git --version
```

Настоящее занятие выполняется на основе решения «Бюро переводов», созданного в ходе практического занятия 1. При отсутствии такого решения допускается использование любого решения на платформе .NET 10.0, содержащего проект Web-приложения и проект модульных тестов.

При недостатке учебного времени допускается отказ от шага 2: конвейер в этом случае составляется без шага выполнения модульных тестов.

Пошаговая инструкция выполнения

Шаг 1. Подготовка решения

1. Откройте терминал в каталоге решения «Бюро переводов».
2. Убедитесь в успешности сборки решения.

```
dotnet build
```

3. Создайте типовый файл исключений системы управления версиями.

```
dotnet new gitignore
```

Указанная команда создаёт файл `.gitignore`, исключающий из репозитория каталоги промежуточных результатов сборки `bin` и `obj`, каталог параметров среды разработки `.vs`, а также файлы пользовательских настроек. Размещение перечисленных объектов в репозитории недопустимо: они формируются заново при каждой сборке и составляют значительный объём.

4. Дополните файл `.gitignore` следующими строками, исключающими файлы базы данных и каталог публикации.

```
# Результаты публикации  
publish/  
  
# Локальные файлы базы данных  
*.db  
*.db-shm  
*.db-wal
```

```
*.mdf
*.ldf
```

- Убедитесь в том, что строка подключения к базе данных не содержит пароля. При применении проверки подлинности средствами операционной системы (`Trusted_Connection=True`) размещение файла `appsettings.json` в репозитории является допустимым. При наличии пароля его следует вынести в хранилище секретов средствами команды `dotnet user-secrets`.

Шаг 2. Создание проекта модульных тестов

Наличие модульных тестов является необходимым условием осмысленности конвейера непрерывной интеграции: конвейер, выполняющий исключительно сборку, обнаруживает лишь ошибки компиляции.

- Создайте проект модульных тестов и добавьте его в решение.

```
dotnet new xunit -n TranslationBureau.Tests -f net10.0
dotnet sln add TranslationBureau.Tests/TranslationBureau.Tests.csproj
```

- Установите ссылку тестового проекта на проект, содержащий проверяемую логику. Наименование проекта зависит от варианта, выполненного в ходе практического занятия 1.

```
:: Вариант 1 (монолитная архитектура)
dotnet add TranslationBureau.Tests reference TranslationBureau

:: Вариант 2 (слоистая архитектура)
dotnet add TranslationBureau.Tests reference TranslationBureau.BusinessLogic

:: Вариант 3 (чистая архитектура)
dotnet add TranslationBureau.Tests reference TranslationBureau.Domain
```

- Замените содержимое файла `TranslationBureau.Tests/UnitTest1.cs` приведённым ниже кодом. Ниже приводится вариант для решения, выполненного по варианту 3 (чистая архитектура); для прочих вариантов проверяются соответствующие им методы проверки правил предметной области.

Файл `TranslationBureau.Tests/TranslatorTests.cs`:

```
using TranslationBureau.Domain.Entities;
using TranslationBureau.Domain.Exceptions;
using Xunit;

namespace TranslationBureau.Tests;

public class TranslatorTests
{
    [Fact]
    public void Create_ПриКорректныхДанных_СоздаётАктивногоПереводчика()
    {
        var translator = Translator.Create("Иванова Мария Петровна",
            "+375291234567", "ivanova@example.com", "высшая", 22m);

        Assert.Equal("Иванова Мария Петровна", translator.FullName);
        Assert.Equal(22m, translator.RatePerUnit);
        Assert.True(translator.IsActive);
    }

    [Fact]
    public void Create_ПриПустомЗначенииФИО_ВозбуждаетИсключение()
    {
        Assert.Throws<DomainException>(() =>
            Translator.Create(" ", null, null, "первая", 18m));
    }

    [Theory]
```

```

[InlineData(0)]
[InlineData(-1)]
public void Create_ПриНеположительнойСтавке_ВозбуждаетИсключение(decimal rate)
{
    Assert.Throws<DomainException>(() =>
        Translator.Create("Петров Сергей Николаевич", null, null, "первая", rate));
}

[Fact]
public void Deactivate_ПереводитЗаписьВАрхивноеСостояние()
{
    var translator = Translator.Create("Сидорова Анна Игоревна",
        null, null, "вторая", 15m);

    translator.Deactivate();

    Assert.False(translator.IsActive);
}
}

```

9. Удалите файл `UnitTest1.cs`, если он был создан шаблоном проекта.
10. Убедитесь в успешном выполнении тестов.

```
dotnet test
```

В окне терминала должно быть выведено сообщение об успешном выполнении всех тестов.

Шаг 3. Локальная сборка и публикация приложения

Прежде чем поручать сборку службе непрерывной интеграции, следует убедиться в её успешности на рабочем месте.

11. Выполните сборку решения в конфигурации `Release`.

```
dotnet build --configuration Release
```

12. Выполните публикацию Web-приложения. Наименование проекта зависит от варианта, выполненного в ходе практического занятия 1: `TranslationBureau` для варианта 1 и `TranslationBureau.Web` для вариантов 2 и 3.

```
dotnet publish TranslationBureau.Web --configuration Release --output ./publish
```

13. Изучите содержимое каталога `publish`. В нём размещены сборки приложения, файл параметров `appsettings.json`, каталог статических файлов `wwwroot` и исполняемый файл.

Следует чётко различать назначение команд сборки и публикации. Команда `dotnet build` выполняет компиляцию исходного кода и размещает результат в каталоге `bin`; полученный набор файлов предназначен для отладки и не является полным. Команда `dotnet publish` формирует набор файлов, достаточный для развёртывания приложения на сервере: она включает файлы зависимостей, файлы конфигурации и статические файлы.

14. Выполните публикацию в виде самодостаточного развёртывания для платформы Windows.

```
dotnet publish TranslationBureau.Web --configuration Release ^
--runtime win-x64 --self-contained true --output ./publish-win-x64
```

Развёртывание, зависящее от платформы (используемое по умолчанию), требует наличия на целевом компьютере установленной среды выполнения .NET соответствующей версии, однако имеет существенно меньший объём. Самодостаточное развёртывание включает среду выполнения в состав пакета, вследствие чего оно не предъявляет требований к целевому компьютеру, однако его объём

составляет несколько десятков мегабайт. Выбор способа определяется условиями эксплуатации приложения.

15. Сравните объёмы полученных каталогов и сделайте вывод о применимости каждого способа развёртывания.

Шаг 4. Создание локального репозитория

16. Инициализируйте репозиторий в каталоге решения.

```
git init
git branch -M main
```

17. Задайте сведения об авторе изменений, если они не были заданы ранее.

```
git config user.name "Фамилия Имя"
git config user.email "адрес@электронной.почты"
```

18. Проверьте перечень файлов, подлежащих включению в репозиторий.

```
git status
```

Убедитесь в том, что каталоги `bin`, `obj` и `publish` в перечне отсутствуют. Их наличие означает неверную настройку файла `.gitignore`.

19. Создайте первую фиксацию изменений.

```
git add .
git commit -m "Решение «Бюро переводов»: исходный код и модульные тесты"
```

Шаг 5. Размещение решения в репозитории GitHub

20. Откройте портал GitHub и создайте новый репозиторий с наименованием `TranslationBureau`.

21. При создании репозитория не устанавливайте флажки добавления файлов `README.md`, `.gitignore` и лицензии: указанные файлы уже присутствуют в локальном репозитории либо будут созданы далее. Создание репозитория с этими файлами приведёт к расхождению историй изменений.

22. Свяжите локальный репозиторий с удалённым, подставив собственное имя учётной записи.

```
git remote add origin https://github.com/ИМЯ_УЧЁТНОЙ_ЗАПИСИ/TranslationBureau.git
```

23. Отправьте изменения в удалённый репозиторий.

```
git push -u origin main
```

24. Обновите страницу репозитория в браузере и убедитесь в том, что исходный код размещён.

Если при выполнении отправки выводится сообщение `failed to push some refs`, это означает, что удалённый репозиторий содержит фиксации изменений, отсутствующие в локальном репозитории. Устранение выполняется следующей командой с последующим повторением отправки.

```
git pull --rebase origin main
```

Шаг 6. Составление описания конвейера непрерывной интеграции

25. Создайте в корне решения каталог `.github/workflows`.

```
mkdir .github\workflows
```

В операционных системах семейства Linux и macOS применяется команда `mkdir -p .github/workflows`.

26. Создайте в указанном каталоге файл `build.yml` со следующим содержимым.

Файл `.github/workflows/build.yml`:

```
# Наименование конвейера, отображаемое в интерфейсе GitHub Actions
name: Build, Test and Publish

# События, при наступлении которых конвейер запускается
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:      # позволяет запускать конвейер вручную

jobs:
  build:
    # Наименование задания формируется с указанием операционной системы
    name: Сборка под управлением ${ matrix.os }
    runs-on: ${ matrix.os }

    strategy:
      # Отказ одного задания не прекращает выполнение остальных
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]

    steps:
      # Шаг 1. Получение исходного кода из репозитория
      - name: Получение исходного кода
        uses: actions/checkout@v4

      # Шаг 2. Установка пакета средств разработки .NET
      - name: Установка .NET SDK
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'

      # Шаг 3. Кэширование пакетов NuGet для сокращения времени сборки
      - name: Кэширование пакетов NuGet
        uses: actions/cache@v4
        with:
          path: ~/.nuget/packages
          key: ${ runner.os }-nuget-${ hashFiles('**/*.csproj') }
          restore-keys: ${ runner.os }-nuget-

      # Шаг 4. Восстановление зависимостей решения
      - name: Восстановление зависимостей
        run: dotnet restore

      # Шаг 5. Сборка решения в конфигурации Release
      - name: Сборка решения
        run: dotnet build --configuration Release --no-restore

      # Шаг 6. Выполнение модульных тестов
      - name: Выполнение модульных тестов
        run: dotnet test --configuration Release --no-build --verbosity normal

      # Шаг 7. Публикация Web-приложения
      - name: Публикация приложения
        run: dotnet publish TranslationBureau.Web --configuration Release --no-build --output ./publish

      # Шаг 8. Сохранение результата публикации в качестве артефакта
      - name: Сохранение артефакта сборки
        uses: actions/upload-artifact@v4
        with:
          name: TranslationBureau-${ matrix.os }
          path: ./publish/**
          retention-days: 30
```

27. Обратите внимание на следующие существенные обстоятельства.

Параметр `strategy.matrix` задаёт перечень конфигураций, для каждой из которых задание выполняется независимо. В приведённом описании сборка выполняется под управлением операционных систем Linux и Windows, что позволяет убедиться в переносимости приложения.

Параметр `--no-restore` шага сборки указывает на то, что восстановление зависимостей уже выполнено предшествующим шагом; параметр `--no-build` шагов выполнения тестов и публикации указывает на использование результатов ранее выполненной сборки. Отказ от указанных параметров приводит к многократному повторению одних и тех же действий и существенному увеличению времени выполнения конвейера.

Шаг кэширования пакетов сохраняет каталог загруженных пакетов между запусками конвейера. Ключ кэша формируется на основании содержимого файлов проектов: при изменении состава пакетов кэш формируется заново.

Событие `workflow_dispatch` обеспечивает возможность запуска конвейера вручную из интерфейса портала, что удобно при отладке описания.

28. Для решения, выполненного по варианту 1 (монолитная архитектура), в шаге публикации указывается наименование проекта `TranslationBureau` вместо `TranslationBureau.Web`.

29. Отправьте описание конвейера в удалённый репозиторий.

```
git add .github/workflows/build.yml
git commit -m "Добавлено описание конвейера непрерывной интеграции"
git push origin main
```

Шаг 7. Проверка выполнения конвейера

30. Откройте страницу репозитория на портале GitHub и перейдите на вкладку «Actions».

31. Убедитесь в том, что запуск конвейера `Build, Test and Publish` начат автоматически.

32. Откройте выполняющийся запуск и изучите протокол каждого шага обоих заданий.

33. Дождитесь завершения выполнения. Успешное завершение отмечается значком зелёного цвета, неуспешное — значком красного цвета.

34. В нижней части страницы запуска в разделе «Artifacts» обнаружьте сформированные пакеты развёртывания `TranslationBureau-ubuntu-latest` и `TranslationBureau-windows-latest`. Загрузите один из них и убедитесь в наличии в нём файлов приложения.

35. Выполните проверку обнаружения ошибок конвейером. Для этого внесите в один из модульных тестов заведомо неверное ожидаемое значение, например замените в тесте `Create_ПриКорректныхДанных_СоздаётАктивногоПереводчика` значение `22m` на `25m`, и отправьте изменение в репозиторий.

```
git add .
git commit -m "Проверка обнаружения ошибки конвейером"
git push origin main
```

36. Убедитесь в том, что конвейер завершился неуспешно, а в протоколе шага выполнения модульных тестов указаны наименование не пройденного теста, ожидавшееся и полученное значения.

37. Восстановите верное значение, отправьте изменение в репозиторий и убедитесь в успешном завершении конвейера.

Шаг 8. Добавление значка состояния сборки

38. Создайте в корне решения файл `README.md` со следующим содержимым, подставив собственное имя учётной записи.

Файл README.md:

```
# Бюро переводов

![Build
Status](https://github.com/ИМЯ_УЧЁТНОЙ_ЗАПИСИ/TranslationBureau/actions/workflows/build.yml/badge.svg)

Web-приложение баз данных «Бюро переводов», разработанное в ходе изучения
дисциплины «Разработка приложений баз данных для информационных систем».

## Состав решения

| Проект | Назначение |
| --- | --- |
| `TranslationBureau.Domain` | доменное ядро |
| `TranslationBureau.Application` | сценарии использования |
| `TranslationBureau.Infrastructure` | инфраструктура и доступ к данным |
| `TranslationBureau.Web` | Web-приложение ASP.NET Core MVC |
| `TranslationBureau.Tests` | модульные тесты |

## Требования

Для сборки и запуска требуются пакет .NET 10.0 SDK и система управления
базами данных MS SQL Server (допускается использование LocalDB).

## Запуск

dotnet run --project TranslationBureau.Web

## Выполнение модульных тестов

dotnet test
```

Состав таблицы приводится в соответствие с вариантом, выполненным в ходе практического занятия 1.

39. Значок состояния сборки может быть получен непосредственно из интерфейса портала. Для этого следует открыть вкладку «Actions», выбрать конвейер в перечне слева, нажать кнопку с тремя точками в правой части страницы и выбрать пункт «Create status badge», после чего скопировать предложенный текст разметки.

40. Отправьте файл в удалённый репозиторий.

```
git add README.md
git commit -m "Добавлено описание проекта и значок состояния сборки"
git push origin main
```

41. Обновите титульную страницу репозитория и убедитесь в том, что значок состояния сборки отображается и содержит надпись **passing**.

Задание для самостоятельного выполнения

Обязательная часть

Обязательная часть задания включает следующие пункты.

42. Подготовить решение к размещению в репозитории: создать и дополнить файл **.gitignore**, убедиться в отсутствии в репозитории промежуточных результатов сборки.

43. Создать проект модульных тестов, реализовать не менее четырёх тестов и убедиться в их успешном выполнении.

44. Выполнить локальную сборку и публикацию приложения, сравнить объёмы пакетов при развёртывании, зависящем от платформы, и при самодостаточном развёртывании.

45. Разместить решение в удалённом репозитории GitHub.
46. Составить описание конвейера непрерывной интеграции, обеспечивающего восстановление зависимостей, сборку, выполнение тестов и публикацию под управлением двух операционных систем.
47. Продемонстрировать обнаружение конвейером намеренно внесённой ошибки и последующее успешное завершение конвейера после её устранения.
48. Добавить файл `README.md` со значком состояния сборки.

Дополнительная часть

Дополнительная часть задания предназначена для обучающихся, завершивших обязательную часть ранее отведённого времени.

49. Дополнить конвейер шагом формирования отчёта о покрытии кода тестами с применением пакета `coverlet.collector` и действия публикации отчёта.
50. Настроить формирование самодостаточных пакетов развёртывания для платформ `win-x64` и `linux-x64` с сохранением каждого в качестве отдельного артефакта.
51. Настроить автоматическое создание выпуска (release) при добавлении в репозиторий тега, наименование которого начинается с символа `v`, с прикреплением к выпуску сформированных пакетов развёртывания.
52. Составить файл `Dockerfile`, обеспечивающий сборку образа контейнера с Web-приложением, и дополнить конвейер шагом сборки образа.
53. Зафиксировать версию пакета средств разработки посредством файла `global.json`, размещённого в корне решения, и пояснить назначение такой фиксации.

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  }
}
```

54. Настроить защиту основной ветви репозитория, установив требование успешного завершения конвейера в качестве условия объединения запросов на включение изменений.

Типичные ошибки и способы их устранения

Ниже перечислены затруднения, наиболее часто возникающие при выполнении настоящего занятия, и способы их устранения.

- **Конвейер не запускается.** Следует проверить путь размещения файла описания: он должен в точности соответствовать `.github/workflows/build.yml`. Следует также убедиться в том, что файл отправлен в ту ветвь, которая указана в разделе `on`, и в отсутствии ошибок синтаксиса языка YAML. Язык YAML чувствителен к отступам, причём символы табуляции в нём недопустимы.
- **Неуспешное завершение шага восстановления зависимостей.** Наиболее частой причиной является несоответствие версии пакета средств разработки, указанной в разделе `setup-dotnet`, целевой платформе проектов. Следует убедиться в том, что указано значение `10.0.x`.
- **Неуспешное завершение шага сборки при успешной сборке на рабочем месте.** Причиной, как правило, является размещение в репозитории каталогов `bin` и `obj`, содержащих файлы, сформированные для иной операционной системы. Указанные каталоги следует удалить из репозитория командой `git rm -r --cached bin obj` и убедиться в корректности файла `.gitignore`.
- **Неуспешное завершение шага выполнения тестов, обращающихся к базе данных.** Виртуальная машина сборки не содержит установленной системы управления базами данных. Модульные тесты не должны обращаться к базе данных; при необходимости проверки взаимодействия с хранилищем

применяется поставщик `Microsoft.EntityFrameworkCore.InMemory` либо система SQLite в режиме размещения базы данных в оперативной памяти.

- **Отсутствие артефакта при успешном завершении конвейера.** Следует проверить значение параметра `path` шага сохранения артефакта: указанный в нём путь должен соответствовать значению параметра `--output` шага публикации.
- **Отправка в репозиторий сведений, составляющих тайну.** Пароли, ключи доступа и строки подключения, содержащие учётные данные, недопустимо размещать в репозитории. Такие сведения задаются в разделе «Secrets and variables» настроек репозитория и передаются конвейеру посредством выражения `${{ secrets.ИМЯ_СЕКРЕТА }}`.